ResuMind: AI Resume Analyzer

A Major Project Synopsis Submitted to

Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal
Towards Partial Fulfillment for the Award of

Bachelor of Technology
(Computer Science and Engineering)

Under the Supervision of

Prof. Krupi Saraf

Submitted By

Devansh Pipraiya [0827CS221083]
Harsh Verma [0827CS221103]
Hardeep Kumar [0827CS221099]

Department of Computer Science and Engineering
Acropolis Institute of Technology & Research, Indore
February-June 2026

1. Introduction of the Project

The job market has become increasingly competitive, and candidates often struggle to understand why their resumes get rejected by Applicant Tracking Systems (ATS) before a human ever reads them. Most people have no way to evaluate how well their resume matches a specific job description, or what improvements would actually make a difference.

This project addresses that gap by building an AI-powered Resume Analyzer that lets users upload their resumes, paste a job description, and instantly receive an ATS compatibility score along with specific, actionable feedback. The system uses AI to evaluate how well a candidate's profile matches the role they are applying for, and returns structured suggestions to help them improve their chances.

Problems Addressed:

- **ATS Rejection Without Feedback** Most ATS platforms reject resumes silently. Candidates receive no explanation of what went wrong or what keywords were missing. This leaves applicants guessing and submitting the same ineffective resume repeatedly across dozens of applications.

- **Lack of Job-Specific Resume Feedback** Generic resume tips found online do not account for the specific requirements of a job posting. A software engineer applying to a data science role needs different feedback than one applying to a backend engineering position. Existing tools rarely tailor their analysis to the actual job listing.

- **No Centralized Resume Storage** Candidates manage multiple versions of their resume across different devices and email threads. There is no simple, browser-based solution that lets them upload, store, and revisit all their resume versions in one place without requiring a backend or account setup.

2. Objective

This project aims to build a browser-based AI resume analysis tool that helps job seekers understand how their resume performs against a specific job listing. The system will extract text from uploaded PDF resumes, compare the content against a provided job description using an AI model, and return a structured ATS score with detailed, job-specific feedback.

The platform will also allow users to store multiple resumes in one place, re-analyze them against different job postings, and track what feedback they have received over time. All of this runs entirely in the browser using Puter.js, meaning there is no backend server, no database setup, and no API key management required from the user.

3. Scope

This project covers the development of a fully client-side web application built with React and React Router v7. The scope includes the following:

Users can sign in and authenticate entirely through Puter.js without any backend. They can upload PDF resumes, which are stored in Puter's cloud file system. They can submit a job description alongside a selected resume and receive an AI-generated analysis that includes an ATS score, matched skills, missing keywords, and improvement suggestions. The application

stores past analysis results so users can review previous feedback. The interface is fully responsive and works across desktop and mobile browsers.

The project does not cover building a backend server, a custom ML model, or integration with actual job portals. The AI analysis is powered by Puter.js which internally routes requests to large language models. Resume parsing is handled client-side through PDF text extraction.

4. Study of Existing System

Resumeworded

- Problems Addressed: Provides ATS score and keyword analysis for uploaded resumes against job descriptions.
- Advantages: Detailed scoring, line-by-line resume feedback, LinkedIn profile review.
- Disadvantages: Requires account creation, limited free tier, no file storage across sessions.
- Gaps Identified: Feedback is not deeply personalized to the specific job description provided. No in-browser storage of resumes.

Jobscan

- Problems Addressed: Compares resumes to job descriptions and highlights missing keywords for ATS optimization.
- Advantages: Side-by-side comparison view, good keyword matching accuracy.
- Disadvantages: Paid subscription for full features, no resume storage, requires repeated uploads.
- Gaps Identified: No AI-generated written feedback. Only keyword matching, no contextual suggestions.

Kickresume

- Problems Addressed: Helps users build and review resumes using templates and AI writing assistance.
- Advantages: Good UI, resume builder included, AI suggestions for phrasing.
- Disadvantages: Focused on resume creation rather than job-specific analysis. No ATS score against a real job posting.
- Gaps Identified: Cannot analyze an existing resume against a specific job description. No client-side storage.

Zety Resume Checker

- Problems Addressed: Reviews resumes for formatting, structure, and content completeness.
- Advantages: Easy to use, highlights structural issues.
- Disadvantages: Analysis is generic and not tied to any specific job description. No ATS simulation.
- Gaps Identified: No job-specific feedback, no resume storage, no AI-generated improvement suggestions.

5. Project Description

ResuMind is a client-side web application that allows users to upload their resumes, store them in the browser using Puter.js, and analyze them against any job description using AI. The entire application runs in the browser with no backend required.

When a user opens the application, they are authenticated through Puter.js using its built-in auth system. Once logged in, they can upload PDF resumes which are stored in Puter's cloud file system under their account. The application displays all uploaded resumes in a dashboard view.

To perform an analysis, the user selects a resume, pastes a job description, and submits the form. The application extracts text from the PDF on the client side, constructs a structured prompt, and sends it to an AI model through the Puter.js AI API. The AI returns a JSON response containing an ATS compatibility score, a list of matched skills, missing keywords, experience feedback, and specific suggestions for improvement.

The analysis result is displayed in a structured UI showing the score prominently, followed by categorized feedback sections. Past analysis results are saved so users can return to them later without re-running the analysis.

The project is built with React 19 and React Router v7 in framework mode, styled with Tailwind CSS, and written entirely in TypeScript. It is containerized with Docker and can be run locally with a single npm command.

6. Methodology/Planning of the Project work

The project is developed using an iterative approach, moving from foundational setup through feature development to testing and deployment. The key phases are:

- **Requirement Analysis:** Research was conducted on how ATS systems evaluate resumes, what structured feedback job seekers need, and how Puter.js handles auth, file storage, and AI API calls. Core features were identified and prioritized based on user value and technical feasibility.

- **Design Phase:** The UI was designed around a clean dashboard layout with clear upload and analysis flows. The data structures for resume metadata and analysis results were planned. React Router v7 route architecture was mapped out, covering the home route, resume detail route, and analysis result route.

- **Development Phase:** The front end was built using React 19 with React Router v7 in framework mode, with Tailwind CSS for styling and TypeScript throughout. Puter.js was integrated for authentication, file storage, and AI API access. PDF text extraction was implemented client-side. The AI prompt was engineered to return structured JSON with consistent fields for the result UI to consume.

- **Testing and Validation:** Components were tested individually. End-to-end flows were validated including upload, storage, retrieval, AI analysis, and result display. Edge cases such as malformed PDFs, empty job descriptions, and AI response parsing errors were handled.

- **Deployment and Maintenance:** The application is containerized using Docker. It can be deployed to any static host or container platform. Post-deployment, the prompt can be updated to improve AI output quality without changes to the core application logic.

Gantt Chart:

Phase	Duration (Weeks)	Milestones
Requirement Analysis	**2**	**Research completed**
Design Phase	**3**	**UI/UX and system design complete**
Development Phase	**6**	**Front-end and back-end finished**
Testing & Validation	**3**	**Platform successfully tested**
Deployment	**2**	**Platform launched**
Variations	**3**	**Platform launched**

7. Features

- **User Authentication via Puter.js:** Users sign in directly in the browser through Puter.js. No backend, no custom auth logic, and no API keys are needed. Authentication state persists across sessions automatically.

- **Resume Upload and Cloud Storage:** Users can upload PDF resumes which are stored in Puter's cloud file system. All uploaded resumes are listed in a dashboard and can be accessed, re-selected, or deleted at any time.

- **AI Resume Analysis:** The core feature of the application. The user selects a resume and provides a job description. The system extracts the resume text, constructs a structured prompt, and queries the AI through Puter.js. The AI returns an ATS score, matched skills, missing keywords, and written improvement suggestions tailored to the specific job description.

- **Structured Analysis Results:** Results are displayed in a clean, organized layout showing the ATS score, a skills match summary, keyword gaps, and section-by-section improvement tips. The response structure is consistent and validated before rendering.

- **Resume History and Past Analyses:** Analysis results are stored alongside resume metadata. Users can return to previous analyses without re-uploading or re-running the AI query.

- **Responsive and Reusable UI:** The interface is built with Tailwind CSS and structured around reusable components. It works across desktop and mobile screen sizes without any layout issues

8. System architecture

ResuMind follows a fully client-side architecture with no dedicated backend server. All data flows through Puter.js APIs running in the browser.

- **Authentication Layer:** Puter.js provides browser-based authentication. On app load, the system checks for an existing Puter session. If none exists, the user is prompted to sign in. The authenticated user object is used throughout the session.

- **File Storage Layer:** Uploaded PDF files are sent to Puter's cloud file system using the Puter.js file API. The application stores file references and metadata in Puter's key-value store, which persists across sessions under the user's account.

- **AI Processing Layer:** When an analysis is requested, the client extracts raw text from the PDF using a client-side parser. The text is combined with the job description into a structured prompt. This prompt is sent to the AI model via the Puter.js AI API. The response is parsed from JSON and stored alongside the resume record.

- **Routing and State:** React Router v7 in framework mode handles client-side routing. Application state is managed using React's built-in hooks. Route loaders are used to fetch resume data and analysis results before rendering each page.

- **Result Storage:** Analysis results are saved back to Puter's key-value store, keyed by resume ID. This allows the dashboard to display past results without repeating AI calls.

9. User Interface (UI)

Dashboard:

- Resume List Panel: Shows all uploaded resumes with name, upload date, and a quick action to analyze or delete.
- Upload Button: Opens a file picker to upload a new PDF resume.
- Recent Analyses: Shows the most recent analysis results with ATS scores at a glance.

Resume Detail View:

- Resume Preview Panel: Displays the extracted text from the selected resume.
- Job Description Input: A textarea where the user pastes the target job description.
- Analyze Button: Triggers the AI analysis flow.

Analysis Result View:

- ATS Score Display: A prominent score indicator showing the compatibility percentage.
- Matched Skills Section: Lists skills from the resume that match the job description.
- Missing Keywords Section: Shows keywords found in the job description but absent from the resume.
- Improvement Suggestions: AI-generated, section-specific suggestions written in plain language.

Wireframes / Mockups:

- Dashboard Wireframe: Header with app logo and user info. Main panel with resume cards and upload CTA. Side navigation for quick access to past analyses.
- Resume Detail Wireframe: Two-column layout with resume text on the left and job description input on the right. Analyze button below both panels.
- Result Wireframe: Score card at the top. Three sections below: matched skills, missing keywords, and suggestions. Back button to return to resume detail.

10. Technology Stack

Front-End Technologies:

- React 19: Component-based UI library for building the interactive interface.
- React Router v7: Framework-mode routing with loaders, actions, and nested routes.
- Tailwind CSS: Utility-first CSS framework for responsive, consistent styling.
- TypeScript: Static typing across all components, routes, and utility functions.
- Vite: Fast build tool and dev server used for local development and production builds.

AI and Platform:

- Puter.js: Client-side SDK providing authentication, cloud file storage, key-value data store, and AI API access. No backend required.
- AI Model via Puter: The AI analysis is powered by large language models accessible through the Puter.js AI API, which handles model routing and cost internally.

PDF Processing:

- Client-side PDF text extraction is used to pull raw text from uploaded PDF files directly in the browser before sending to the AI.

DevOps:

- Git and GitHub: Version control and source code management.

Development Tools:

- Visual Studio Code: Primary development environment.
- npm: Package management and script running.
- react-router.config.ts and vite.config.ts: Configuration files for routing and build setup.

11. Testing Plan

Unit Testing:

- Objective: Validate individual components and utility functions in isolation.
- Method: Test PDF text extraction, prompt construction, JSON response parsing, and individual UI components.
- Outcome: Each module works correctly before being integrated into the full flow.

Integration Testing:

- Objective: Verify that Puter.js auth, file storage, and AI API calls work together correctly.
- Method: Simulate a full user session — sign in, upload a file, trigger analysis, retrieve results — and verify data flows correctly at each step.
- Outcome: End-to-end data flow between components and Puter.js APIs works without errors.

Functional Testing:

- Objective: Confirm the application behaves correctly from a user perspective.
- Method: Manual testing of all user-facing flows including login, upload, job description input, analysis trigger, and result rendering.
- Outcome: All user flows complete without errors or unexpected behavior.

User Acceptance Testing (UAT):

- Objective: Validate that the application is useful and easy to use for real job seekers.
- Method: Have test users upload their own resumes against real job postings and collect feedback on the quality of AI analysis and the usability of the interface.
- Outcome: Confirm the ATS score and feedback are meaningful, accurate, and actionable.

Performance Testing:

- Objective: Ensure the application handles file uploads and AI response times within acceptable limits.
- Method: Test with large PDF files, slow network conditions, and back-to-back analysis requests.
- Outcome: Application remains responsive and handles edge cases gracefully.

Security Testing:

- Objective: Ensure user data is handled safely.
- Method: Verify that resume files and analysis results are only accessible to the authenticated Puter user. Confirm no data is leaked between sessions.
- Outcome: Each user's data remains private and isolated within their Puter account.

Regression Testing:

- Objective: Ensure new changes do not break existing functionality.
- Method: Re-run all functional and integration tests after each significant code change.
- Outcome: Application remains stable throughout the development cycle.

12. Expected Outcome

The completed ResuMind application will give job seekers a practical, free, and easy-to-use tool to understand exactly how their resume compares to a specific job description. Rather than generic advice, users will receive an ATS score and targeted feedback that reflects the actual requirements of the role they are applying for.

By running entirely in the browser with no backend setup, the application is accessible to anyone without technical knowledge. Users do not need to manage accounts on a separate server, pay for a subscription, or deal with complex setup. The Puter.js platform handles auth, storage, and AI access transparently.

The expected outcomes include: job seekers making more targeted resume edits before applying, fewer applications being silently rejected by ATS, and users developing a clearer understanding of how keyword matching works in recruitment. Over time, the stored resume history and past analyses will help users track their improvements across different job applications.

13. Resources and Limitations

Resources Required:

- **Hardware:** A development machine capable of running Node.js, npm, and Docker. A modern browser for testing client-side Puter.js functionality.
- **Software:** Node.js, npm, Git, Docker, Visual Studio Code, and a modern browser. All other services (auth, storage, AI) are provided by Puter.js at no cost.
- **Data:** Real PDF resumes and real job descriptions for testing. No proprietary dataset is required since AI processing is handled by Puter.js.
- **Human Resources:** A team of developers familiar with React, TypeScript, and React Router v7. No backend engineers or DevOps specialists are required given the serverless architecture.

Limitations:

- **AI Response Quality:** The quality of analysis depends on the AI model accessed through Puter.js. Responses may occasionally be inconsistent in format, requiring robust JSON parsing and fallback handling.
- **PDF Parsing Accuracy:** Client-side PDF text extraction may not handle all PDF formats correctly, particularly scanned or image-based PDFs where text is not machine-readable.
- **Puter.js Dependency:** The entire platform depends on Puter.js remaining available and maintaining its current API. Changes to Puter's pricing or API structure could affect the application.
- **No Custom Model Training:** The AI has not been trained specifically on resume data. Analysis quality is limited to what the general-purpose language model can infer from the prompt alone.
- **Offline Use:** The application requires an internet connection to authenticate, load files, and run AI analysis. No offline mode is available.

14. Conclusion

ResuMind demonstrates how a fully functional, AI-powered web application can be built without any backend infrastructure. By combining React, React Router v7, TypeScript, and Puter.js, the project delivers a practical tool that gives job seekers real, job-specific feedback on their resumes in seconds.

The project addresses a genuine problem — the opacity of ATS systems and the lack of personalized resume feedback — and solves it with a clean, accessible, client-side application. The use of Puter.js as the platform layer removes the usual complexity of auth, storage, and AI integration, making the codebase lean and easy to maintain.

Beyond the technical implementation, the project shows how modern browser capabilities and AI APIs can be combined to build tools that were previously only possible with significant backend infrastructure. The same architecture could be applied to other document analysis tasks, making this a useful reference for serverless AI application development.

15. References

1. Puter.js Documentation, "Puter JavaScript SDK," [Online]. Available: https://docs.puter.com
2. React Documentation, "React 19 Release," [Online]. Available: https://react.dev
3. React Router Documentation, "React Router v7 Framework Mode," [Online]. Available: https://reactrouter.com/home
4. Vite Documentation, "Getting Started," [Online]. Available: https://vite.dev/guide
5. TypeScript Documentation, "TypeScript Handbook," [Online]. Available: https://www.typescriptlang.org/docs
6. R. Kelleher, "How Applicant Tracking Systems Work," SHRM, 2022.